Controlled Generative Development: From Intent to Structural Realization and Preparation for Controlled Execution

Abstract

This paper proposes a new formulation of the problem for AI-native development in which the central object before code generation is neither the prompt nor the code, but the **structural realization** of the task. We describe CGD through the architectural framing **Intent** $\rightarrow$ **Structural Realization** $\rightarrow$ **Execution** and through the top-level pair $\mathbf{D} = \langle \mathbf{I}, \mathbf{R} \rangle$, where I denotes an intent representation and R denotes a structural realization. For CGD, the formal core of the second element is given by the triple $\mathbf{R} = \langle \mathbf{G}, \mathbf{\Sigma}, \mathbf{C} \rangle$: a typed graph of functions and tables, table signatures, and function contracts. This formulation makes it possible to separate the free interface of intent from a strict layer suitable for verification, analysis, modification, and preparation for transfer into execution backends. The central thesis of the paper is that practical controllability in AI-assisted development begins not with ever more elaborate input prompts, but with fixing the **model output** through an output contract, abstraction control, and a verification profile. Across three demonstrations, we show that the output contract makes architectural omissions observable, abstraction control reduces drift in the level of description, and verification-driven dialogue makes it possible to bring a structural realization to specification closure before code generation. The paper is presented as conceptual and architectural: we do not claim a completed industrial implementation, but propose a formal scaffold and explain why it defines the right class of problems for further implementation work.

Keywords: controlled generative development; structural realization; output contract; pre-code verification; verification-driven dialogue; task-driven context assembly; abstraction control; AI-native software design

1 Introduction

Generative models have dramatically reduced the cost of obtaining code, but they have not solved an earlier and more fundamental problem: how to preserve the causal link between **what the author wanted**, **how it was formalized**, and **what is ultimately executed**. In a typical workflow, a user writes a free-form request, the model returns a mixture of text, pseudocode, lists, and diagrams, and the developer manually tries to reconstruct the architecture of the solution. As a result, the cheapest defects to fix—missing branches, absent input artifacts, implicit contracts, and broken data flows—are discovered too late, after they have already been embodied in code.

Modern agentic systems such as OpenAI Symphony (2026) [6] automate the transition from task to code, but in the public descriptions currently available for such systems, no intermediate verifiable layer is singled out between intent and execution. This work addresses precisely that gap.

The problem is not simply that models are "not intelligent enough." The deeper problem is that between intent and code there is often no stable layer suitable for formal work. A free-form model response may be useful to a person as a sketch, but it is inconvenient as an object of analysis: its form drifts, its completeness varies, its level of abstraction is unstable, and many assumptions remain implicit. This is why such a setup makes it difficult to compare solutions reliably, assemble context for a specific change, detect defects before generation, and hand the same specification to different execution backends.

This paper shifts the center of gravity. We do not place at the center the question "which model is better," and we do not search for a single correct graph. Instead, we fix the layer into which a model's response must be brought **before** code generation. We call this layer **structural realization**. It is neither a free-form description of intent nor executable code, but it is precisely the place where one can: verify the task before code generation; make omissions and contradictions observable; assemble relevant context for the next change; compare and correct solutions without reducing the paper to a competition between models; and hand the same specification to different execution backends.

In this sense, the key shift of the work lies not in making the input more elaborate, but in **fixing the output**. We argue that practical progress in AI-assisted development begins not where the user writes increasingly clever prompts, but where the model response is required to fit into a verifiable and controllable structure in which incompleteness becomes observable.

To this end we propose the **Controlled Generative Development (CGD)** framing. At the top level it is described by the chain **Intent** $\rightarrow$ **Structural Realization** $\rightarrow$ **Execution**. Architecturally, before handoff to a backend, a project is described by the pair $\mathbf{D} = \langle \mathbf{I}, \mathbf{R} \rangle$, where I is an intent representation at the interface level and R is the structural realization. For CGD, the formal core of the second element is given by the triple $\mathbf{R} = \langle \mathbf{G}, \mathbf{\Sigma}, \mathbf{C} \rangle$, where G is a typed graph of functions and tables, Σ is a set of table signatures, and C is a set of function contracts.

The main contribution of the paper is neither the claim that we have found the "one correct graph," nor a comparison of specific models against each other. We show something else: if intent is translated into a strict and verifiable structural layer in which omissions become observable, then the mixing of problems disappears. Questions of response form, level of abstraction, completeness of description, data compatibility, and the choice of execution backend no longer live at the same level; they begin to be solved where they properly belong.

2 Three control loops: output contract, abstraction control, verification profile

2.1 Contract-first at the communication level

CGD implements contract-first not only at the level of functions and tables, but also at the level of interaction with the model. The model receives not merely a task, but a **contract on the shape of the response**. In this formulation, the central object becomes not the prompt content itself, but the space of admissible responses into which the model must fit.

This shift matters in practice. As long as the response format is not fixed, the user and the tool work with text meant for reading. Once the response format becomes part of the contract, the response turns into data meant for analysis. A missing table, an unfilled contract, an incomplete error path, or a drifting abstraction level become observable defects rather than something to be guessed at.

2.2 Output contract

The first control loop is the **output contract**. It defines **what** must be present in the model response and **how** it is serialized.

Inside it, it is useful to distinguish two substructures. The first answers the question **what is required**: which components must exist in the response at all. In the CGD context, these are functions, tables, signatures, contracts, and typed relationships. The second answers the question **how it is represented**: which machine-readable and human-readable forms are used to carry the result. For example, one part may be serialized as a structured JSON profile, while another part may be expressed as a human-readable graph projection.

This distinction is fundamental. It makes it possible to avoid conflating the substantive obligations of the response with the concrete format used to represent them. The output contract is not equal to a single "file format"; it combines the mandatory components of the response and the way they are serialized.

The concrete realization of the output contract for CGD—the CGD Response Standard—is given in Appendix A.

2.3 Abstraction control

The second control loop is **abstraction control**. Even when the form of the response is fixed, the model can still vary dramatically in its level of detail.

One response may split a solution into many micro-steps, another may compress everything into a single large node, and a third may mix architectural and operational levels. For this reason, fixing structure and form alone is not enough. One must also constrain the **resolution** of the response: top-level architecture, detailed decomposition, local workflow, internal function step, and so on.

An important effect of abstraction control is that it does not force the model into a single architecture. It constrains only the admissible **level of description**, while leaving the value axis free.

The concrete realization of abstraction control for CGD—the Execution Profile—is given in Appendix B.

2.4 Verification profile

The third control loop is the **verification profile**. It determines which properties of the response are checked before code generation, which defects count as blocking, what may be repaired automatically, and at what point the structural realization may be considered sufficiently complete for transfer to a backend.

The verification profile completes the transition from "text to read" to "structure to work with." The output contract makes the response inspectable, abstraction control makes it comparable in depth, and the verification profile makes it suitable for formal pre-code verification.

2.5 Why these three loops are sufficient for the first version

These three loops do not attempt to prove that an architecture is optimal, nor do they guarantee the semantic correctness of the solution. Their task is more modest, but more fundamental: to eliminate the chaos of mixed levels and translate the model response into a space in which completeness can be observed, consistency can be checked, corrective questions can be asked, and the resulting structure can be reused in further development.

That is why CGD starts not from the search for an ideal model, but from the design of a controllable response space.

2.6 Positioning

By early 2026, the spec-first approach had become a stable industrial trend: academic surveys systematize levels of specification strictness [7], while industrial tools—from CodeSpeak [3] and Augment Intent [2] to OpenAI Symphony [6], Cursor Rules [4], Claude Code [1], and GitHub Spec Kit [5]—treat specification as the primary development artifact rather than as a free-form prompt.

However, all of these systems address primarily one side of the problem: the transition from specification to code. In the publicly available descriptions of these systems, no separate intermediate verifiable layer is singled out in which the specification itself can be checked, structural defects can be detected, and the system can be brought to specification closure **before** code generation. CGD addresses precisely this gap: the formal core $R = \langle G, \Sigma, C \rangle$, the output contract, abstraction control, and the verification profile define a space in which an error is caught before it becomes code.

3 Architectural framing: $D = \langle I, R \rangle$

3.1 Intent and Structural Realization

The top-level CGD scheme is not a flat set of equal components, but an architectural cause $\rightarrow$ consequence pair:

$$D = \langle I, R \rangle.$$

The first element, I, represents **intent**. This is where the user or author describes what needs to be done. In our current implementation, the interface level is served by **YASF (Yet Another Structured Format)**—a minimal author-facing format for describing intent. At this level, the task is fixed in terms of the problem domain and in names that are natural for the author; these names and structures do not yet need to coincide with the future structural realization. YASF is not a formal realization and does not directly define G, Σ, or C; it merely translates a free-form description into a structured interface representation suitable for further compilation. The transition from the first element of the pair to the second consists in compiling a YASF description into the structural realization $R = \langle G, \Sigma, C \rangle$, where the graph, table signatures, and function contracts already obey strict rules of consistency and verification. The full YASF specification is given in Appendix F.

The second element, R, represents the **structural realization** of intent. This is no longer a free-form description, but a formal model that can be processed automatically: verified, analyzed, modified, and handed to an execution system.

3.2 Why the pair is needed but does not replace the formal core

The pair $D = \langle I, R \rangle$ answers the question **where the boundary lies between intent and formalized realization**. It is not meant to express all details of verification. Formal work begins inside the second element.

For CGD, this second element has a concrete internal structure:

$$R = \langle G, \Sigma, C \rangle.$$

The pair therefore does not replace the triple. It puts it in the right place. Without this distinction, it is easy to mix the language of intent, the standards that represent the response, and the realization model itself. In the new formulation, these confusions disappear: D explains the architectural framing, while R defines the formal core of CGD.

3.3 Intent → Structural Realization → Execution

The CGD architectural chain has three links. At the **Intent** level, the task is fixed in an interface form convenient for the author. At the **Structural Realization** level, the intent is translated into formal structures suitable for verification and analysis. At the **Execution** level, the structural realization is handed to an execution system: a code generator, interpreter, simulator, analytical backend, or multi-agent system.

This chain makes it possible to separate three tasks that are often mixed together: describing meaning, formalizing that meaning, and executing it. That separation is the main architectural gain of the new version of the paper.

4 The formal core of CGD: $R = \langle G, \Sigma, C \rangle$

4.1 Graph G

The graph G is defined as a typed directed graph

$$G = (V, E, \mathrm{src}, \mathrm{dst}, \tau_V, \tau_E),$$

where the node type satisfies $\tau_V(v) \in \{\texttt{function}, \texttt{table}\}$, and the edge type satisfies $\tau_E(e) \in \{\texttt{consumes}, \texttt{produces}, \texttt{reads}, \texttt{writes}, \texttt{triggers}\}$.

The basic admissible combinations are: `consumes`: table → function; `produces`: function → table; `reads`: table → function; `writes`: function → table; `triggers`: function → function.

The graph fixes the architecturally meaningful topology of processing: which functions participate in the process, which tables act as inputs, outputs, and states, and which dependencies between steps are trigger dependencies rather than data flows.

4.2 Table signatures Σ

A table signature $\Sigma(t)$ is defined as

$$\Sigma(t) = \langle \mathrm{Fields}, \mathrm{Keys}, \mathrm{Kind}, \mathrm{Constraints} \rangle.$$

Here `Fields` describes fields and their types, `Keys` describes primary, unique, and foreign keys, `Kind` describes the role of the table in the model (`entity`, `event`, `reference`, `log`, `projection`, `error`), and `Constraints` captures additional integrity constraints.

Signatures are not a duplicate of a relational schema for its own sake. They are the source of type information for checking compatibility between graph relations and function contracts. If the graph says that a function produces a table, while the signatures show that the table is incompatible with the expected data structure, that must be detected before code generation.

4.3 Function contracts C

A function contract $C(f)$ is defined as

$$C(f) = \langle C_{\mathrm{in}}, C_{\mathrm{out}}, R, W, T_{\mathrm{in}}, T_{\mathrm{out}} \rangle,$$

where: C_{in} — the tables consumed by the function; C_{out} — the tables created by the function; R — the tables read by the function; W — the tables written by the function; T_{in} — incoming trigger dependencies; T_{out} — outgoing trigger dependencies.

Inside the contract, the symbol R denotes the reads component of a function; in the pair $D = \langle I, R \rangle$, the same symbol denotes the structural realization as a whole. The distinction is given by context.

A function contract links the graph and the signatures: it describes not merely the fact that a `function` node exists, but how that node interacts with the data structure and with other processing steps.

4.4 Root property: verification before code generation

The main property of the triple $\langle G, \Sigma, C \rangle$ is that it makes it possible to catch errors **before** code generation. At this level one can already detect: missing input artifacts; omitted processing branches; broken data flows; incomplete contracts; incompatible table signatures; isolated or unreachable elements; and mixed abstraction levels within one response.

This is the root property of CGD. Structural realization is needed neither for a pretty diagram nor to duplicate code. It is needed because this is the layer where an error is still cheap to fix.

4.5 The triune task: execute, analyze, modify

It is precisely the structural realization—and not intent and not code—that is capable of solving three tasks at once.

Execute. The triple $\langle G, \Sigma, C \rangle$ can be handed to an execution system. **Analyze.** It supports formal checking of consistency, completeness, and compatibility. **Modify.** It supports controlled transformations: changing nodes and links, updating contracts, restructuring data, and shifting the level of detail.

Intent is too free-form to solve all three tasks reliably. Code is too detailed and too tightly bound to a chosen backend. Only the intermediate structural layer is both sufficiently strict and sufficiently compact.

4.6 Specification as code with legibility

Structural realization has one more important property: it behaves like code while remaining more legible than code. It can be versioned, diffed, validated, passed between tools, and used as a source for generation. At the same time, it remains readable at the architectural level: the graph shows topology, signatures show data, and contracts show processing interfaces.

This effect is conveniently described as **specification as code with legibility**.

4.7 Contract-preserving transformations

The triple $\langle G, \Sigma, C \rangle$ supports contract-preserving transformations: local decomposition of a subgraph with preservation of its external contract, and reverse aggregation. For example, three micro-functions with compatible contracts can be collapsed into one macro-function if the external contract of the subgraph matches the contract of that macro-function. For the first version of the paper, what matters is to fix the very existence of such transformations, because this explains why CGD is able not only to store structure, but also to work with different levels of its detail. The full algorithm for computing a subgraph contract and the formal rules for such transformations are moved to Appendix C.

5 Verification-driven dialogue and specification closure

5.1 Verification profile as an operating mechanism

In CGD, the verification profile is not an abstract idea of "checking something before code." It is a concrete operating mechanism that combines automatic checks, defect classification, and targeted dialogue with the author.

As soon as a model response has been brought to the triple $\langle G, \Sigma, C \rangle$, the system is able not only to validate format, but also to ask substantive questions about incompleteness in the structure.

5.2 Classes of defects

Defects are divided into three classes.

Blocking. Defects that cannot be eliminated without a decision from the author. For example, a missing error path, an undefined input artifact, or a fundamentally incomplete function contract.

Auto-resolvable. Defects that can be corrected automatically without changing semantics. For example, name normalization, alias merging according to a predefined rule, canonicalization of type notations, or deletion of structural duplicates.

Advisory. Remarks that do not block progress, but point to a potential problem: unconfirmed redundancy, an optional downstream step left uncovered, an unused success event, incomplete logging, and so on.

This separation matters because it does not mix technical normalization, substantive omissions, and architectural recommendations.

5.3 Verification-driven dialogue

When the verifier detects a blocking defect, it does not attempt to silently "guess on behalf of the author." Instead, it generates a **targeted question**, formulated in natural language. The author's response updates the structural realization, after which verification is launched again.

This loop is what we call **verification-driven dialogue**. It moves verification from the mode "the model produced text and the human noticed the problem by hand" to the mode "the system localizes the defect, asks a question where automatic repair is not admissible, and resumes the process after receiving an answer."

5.4 Specification closure

A specification is considered to have reached **specification closure** when all of the following conditions are satisfied:

1. the structural realization is consistent: graph, signatures, and contracts do not contradict one another;
2. all mandatory contract components are defined;
3. no unresolved blocking defects remain;
4. the mandatory processing paths, including error handling, have explicit structural expression;

5. all advisory remarks have either been eliminated or explicitly accepted as admissible within the current verification profile.

Importantly, specification closure is neither a property of a single level nor a sign that the architecture is "ideal." It is a **sufficient state** for moving to the next stage: code generation, simulation, or another execution-oriented processing step.

6 Task-driven context assembly

In CGD, task-driven context assembly is not performed over the entire chat history and not through general embedding similarity. It is driven by the affected objects of the structural realization. When a new task arrives, the system first localizes it in $R = \langle G, \Sigma, C \rangle$: it finds the functions, tables, contracts, and nearest dependency neighborhood touched by the change. Those very objects are then used as keys for searching for additional information in the history of interaction—deferred decisions, author preferences, disputed areas, and rules that have not yet been fully materialized in the current realization.

For example, if the task is "add error logging for registration," the system locates the registration function, its contract, the tables on the error path, and the nearest logging sink. It then lifts the history for those same objects: it may turn out that two weeks earlier the author already recorded a preference to log errors separately from the success path, but that decision was never introduced into the structure. The resulting context includes the current state of the affected objects, their local neighborhood, and the historical decision that was found. This makes the context compact, current, and anchored to the single source of truth—the structural realization; the full formalization is moved to Appendix D.

7 Demo 1—controlling form

7.1 Setup

The same user registration request is taken twice. In the first run it is submitted without an output contract. In the second run it is submitted with an explicit output contract that requires the response to return functions, tables, signatures, contracts, and a typed projection of relations in a fixed form.

To demonstrate the effect of form, responses from different models are used; comparing models against one another is not the goal of this demo. By contrast, Demo 2 uses the same model in order to isolate the effect of one factor only—the level of abstraction.

7.2 Real baseline fragment

The following is a verbatim fragment from a Claude Opus 4.6 baseline response without a structuring output contract:

```
## Functions

### 1. register_user(email, password) -> User

Creates a user and starts confirmation.

Input: email: str, password: str
Output: User (id, email, is_verified, created_at)
```

```
Errors: ValueError -- email already taken or invalid

Logic:
  1. validate_email(email)
  2. hash = hash_password(password)
  3. INSERT INTO users (email, password_hash)
  4. token = create_verification_token(user.id)
  5. send_verification_email(email, token)
  6. log_action(user.id, 'register', {ip})
  7. return user
```

This fragment is illustrative because it mixes the function interface, logic, errors, and side effects, but contains no explicit separation between graph, data, contract, and downstream dependencies. Omissions remain implicit: one cannot formally check whether a separate input artifact exists, how the error path is materialized, or which tables act as outputs of the processing.

7.3 Real structured fragment

The following are verbatim fragments from a structured GPT-5.4 run with an output contract: first the graph projection, then the contract of the main function.

```
"contract": {
  "Cin": ["RegistrationRequest"],
  "Cout": ["User", "ConfirmationToken", "ActionAuditRequest"],
  "R": ["User"],
  "W": [],
  "Tin": [],
  "Tout": ["SendConfirmationEmail", "LogAction"]
}
```

Here an omission can no longer be hidden in free text: if a function has an empty `Cin`, if `x-kind` is missing, or if the error path exists only "in words," that becomes an observable structural defect rather than a matter of interpretation.

7.4 Takeaway from Demo 1

The point of the first demonstration is not that the output contract makes the model "smarter." It makes the response **workable**. The response ceases to be text for reading and becomes structure for analysis.

8 Demo 2—controlling abstraction

8.1 Setup

The second demo uses the same output contract, but changes the abstraction constraint. For an honest comparison, two responses from the same GPT-5.4 model to the same task are taken: the original structured run without an explicit top-level constraint, and a separate run with an instruction to simplify the graph as much as possible to a top-level representation.

8.2 Real fragment without a level constraint

The following is a verbatim fragment from the original structured GPT-5.4 run without an abstraction-level constraint:

Even this short fragment shows that the model naturally decomposes the solution into several functions and six tables, including distinct confirmation and logging steps.

8.3 Real fragment with a level constraint

The following is a verbatim fragment from a separate GPT-5.4 run with the requirement to simplify the graph as much as possible to a top-level representation:

```
"contract": {
  "Cin": ["RegistrationRequest"],
  "Cout": ["User", "ActionLog"],
  "R": ["User"],
  "W": [],
  "Tin": [],
  "Tout": []
}
```

In the second response, the separate nodes `SendConfirmationEmail`, `LogAction`, `ConfirmationToken`, `EmailDispatch`, and `ActionAuditRequest` disappear from the top-level representation; the corresponding actions remain at the level of semantic description and processing, but the upper structural projection compresses into a single macro-step of registration.

8.4 Takeaway from Demo 2

The second demonstration shows that fixing form alone is not sufficient. To continue working with the structural realization, one must also control the level of abstraction. Output contract and abstraction control work as a complementary pair.

9 Demo 3—verification-driven correction

9.1 Setup

In the third demo, one of the structured responses obtained after Demo 1 and Demo 2 is passed through the verification profile. The goal of the demonstration is not to drive the response to a rigid predefined canon, but to show how structured output becomes an object of verification and correction until specification closure is reached.

The full artifacts of the demo runs are given in Appendix E.

9.2 First verifier pass

In the real GPT-5.4 run, the main handler `RegisterUser` already has an explicit input artifact $C_{\text{in}} = [\text{“RegistrationRequest”}]$, so the defect does not lie in the main registration step. The verifier detects two blocking defects:

1. **Blocking:** the function `SendConfirmationEmail` has an empty C_{in}, even though the email-sending step depends on an input artifact that is not structurally expressed.

2. **Blocking:** the registration flow lacks a materialized error path: the error is mentioned at the semantic level, but receives no separate structural expression in the graph, signatures, and contracts.

Besides the blocking defects, the same run contains auto-resolvable notational deviations: a non-standard form of Mermaid edge labels and values such as `format: "text"` / `format: "bool"` in JSON signatures. These deviations can be normalized automatically and do not require a decision from the author.

9.3 Dialogue with the author

For both blocking defects, the system asks targeted questions: which object should count as the explicit input artifact for `SendConfirmationEmail`; and how the registration error should be materialized in the structural realization: as a separate table, an event, or another explicitly specified sink.

The author's answer updates the triple $\langle G, \Sigma, C \rangle$: an explicit C_{in} is added to the contract of `SendConfirmationEmail`, and the error path receives structural expression with recomputation of the related signatures and contracts.

9.4 Re-check and closure

The expected result of the second verification pass is as follows: after adding an explicit input artifact to the contract of `SendConfirmationEmail` and materializing the error path, both blocking defects can be eliminated. In the current version of the paper, this second pass is not shown as a separate saved artifact; here we fix the structure of the correction and the condition under which specification closure is reached.

9.5 Takeaway from Demo 3

The third demonstration closes the main logic of the paper. The output contract provides structure, abstraction control keeps the description at the intended level, the verification profile localizes defects, and verification-driven dialogue provides a mechanism by which a structural realization can be brought to specification closure. In the current demo, the first verification pass and the structure of the correction are shown; an empirical record of the full cycle up to closure is the nearest next step.

10 Limitations and threats to validity

10.1 Limits of the claims

This work is conceptual and architectural. We do not claim that we have already built a complete reference implementation, nor do we claim demonstrated industrial effectiveness. The paper shows how the problem should be structured rather than presenting a final operational report.

Moreover, the output contract does not guarantee an optimal architecture. It guarantees something else: it makes an architecture **observable, verifiable, and correctable before code generation**.

10.2 Limits of the model

The formal core of CGD primarily covers the class of systems with discrete data flows and explicit dependencies between functions and tables. The full operational semantics of branching, loops, concurrency, timing, and transactionality are not exhausted in this version of the paper.

10.3 Threats to construct validity

The danger here is that the chosen control loops may look sufficient for a broader class of tasks than they actually cover. We reduce this risk by explicitly distinguishing the architectural framing $D = \langle I, R \rangle$ from the formal core $R = \langle G, \Sigma, C \rangle$, and by not extending the claims to every possible class of computational systems.

10.4 Threats to internal validity

Internal validity is limited by the absence of a full reference implementation. The demonstrations shown support the architectural plausibility of the approach, but do not prove that every concrete implementation of a verification profile or task-driven context assembly will necessarily yield the same practical effect.

10.5 Threats to external validity

External validity is limited by the fact that the paper uses tasks that naturally fit a processing graph and data tables. Extending the approach to other classes of systems will require additional representation profiles and possibly an extension of the triple $\langle G, \Sigma, C \rangle$.

10.6 Threats to reliability

The reliability of results depends on the stability of the output contract, the normalization rules, and the verification profile. If these rules remain implicit or drift between runs, the paper loses one of its main advantages—repeatability at the structural level. For this reason, the output contract, the verification profile, and the normalization rules must be versioned on a par with the structural realization itself. This is precisely why we move strictness from the domain of "good prompts" into the domain of explicit profiles and verification rules.

11 Future Work

11.1 Calibrating the output contract and abstraction control

The first natural direction for future work is to calibrate the output contract and abstraction control against fixed structural metrics. Once the form of the response has been standardized, prompt and profile optimization becomes possible against criteria such as completeness, consistency, and suitability for downstream verification.

11.2 Classes of equivalent solutions

The second important task is to define classes of equivalent solutions. For practical work, it is not enough to distinguish "literally identical" from "not identical." Formal criteria are needed to distinguish a different level of detail, an admissible structural transformation, and a genuine architectural divergence.

11.3 Graph transformations and path-based comparison

The third direction is graph transformations and path-based comparison. Several related questions open here at once: admissible collapses and expansions of subgraphs, contract-preserving transformations, comparison of realizations by the set of paths from inputs to outputs, and determination of which differences belong to form and which belong to architectural substance.

11.4 Extending the set of execution backends

The fourth direction concerns the backend level. One and the same structural realization should be suitable not only for code generation, but also for interpretation, simulation, analytics, and agentic execution. Determining which properties of the triple $\langle G, \Sigma, C \rangle$ must remain invariant across such backends is an interesting research problem in its own right.

12 Conclusion

This paper has proposed a new formulation of the problem for AI-native development. Instead of treating prompt and code as two endpoints of the process, we introduce an intermediate layer—**structural realization**—that becomes the main object of verification, analysis, and transfer to a backend.

At the top level, CGD is defined by the architectural chain **Intent** $\rightarrow$ **Structural Realization** $\rightarrow$ **Execution** and by the pair $\mathbf{D} = \langle \mathbf{I}, \mathbf{R} \rangle$. Inside CGD, structural realization is defined by the triple $\mathbf{R} = \langle \mathbf{G}, \mathbf{\Sigma}, \mathbf{C} \rangle$. This separation removes the mixing of levels: the interface-level description of intent, the formal realization layer, and the execution system no longer compete for the same place in the model.

The three control loops—**output contract**, **abstraction control**, and **verification profile**—translate a model response from the space of free text into the space of controlled structure. The output contract makes incompleteness observable, abstraction control keeps the admissible depth of description under control, and the verification profile makes it possible to ask targeted questions and bring the realization to specification closure.

The formal core $\langle G, \Sigma, C \rangle$ provides the root property of CGD: verification before code generation. It is precisely on this layer that the triune task becomes possible—to execute, analyze, and modify one and the same specification. Task-driven context assembly defines the principle according to which this layer is needed not only for verification, but also for assembling relevant context for the next change. Finally, the architectural framing is designed so as to admit backend substitutability: one and the same structural realization is, in principle, suitable for transfer into code generation, interpretation, simulation, or a multi-agent system. Verifying this property in practice is a separate direction of future work (§11.4).

We do not claim that this already solves the problem of correct program-system generation. But we do claim that this way of structuring the problem formulates it correctly. It distributes intent, formalization, and execution across their proper levels and thereby removes the mutual interference of problems that, in ordinary AI-assisted development, end up mixed in one place.

A CGD Response Standard

A.1 Purpose

This standard defines the format of a structured model response when designing software systems within the CGD (Controlled Generative Development) approach. The standard ensures

comparability of responses across models, binary completeness checking, and the possibility of automated verification.

A.2 Principle

The standard implements contract-first at the level of communication with the model. The model receives a contract for the response format before solving the task. A violation of the format is visible without expert interpretation.

A.3 Structure of the response

The response consists of two mandatory sections. Each section begins with a second-level heading (`##`) and contains exactly one fenced block. Text outside these sections is not allowed.

Section	Format	Purpose
Mermaid	`flowchart LR`	Typed directed graph: nodes and edges
CGD Specification	JSON	Full system description: functions and tables

A.4 Section 1: Mermaid

A.4.1 Format

Only `flowchart LR` is allowed. Other Mermaid diagram types (`erDiagram`, `sequenceDiagram`, `classDiagram`) are not allowed.

A.4.2 Node types

Tables are represented by trapezoids with the `tbl_` prefix in the identifier. The display name contains the table name and its `kind` in parentheses:

```
flowchart LR
    tbl_user[/"User (entity)"/]
```

Functions are represented by rounded rectangles with the `fn_` prefix in the identifier:

```
flowchart LR
    fn_register_user(["RegisterUser"])
```

A.4.3 Edge types

Exactly five edge types are allowed. The type is indicated in the edge label.

Type	Direction	Meaning
consumes	table → function	the function takes a record as an input artifact
produces	function → table	the function creates a new record
reads	table → function	the function reads data without changing it
writes	function → table	the function adds or updates a record
triggers	function → function	the function invokes another function

A.4.4 Forbidden combinations

- `table` → `table` is not allowed.
- `function` → `function` is allowed only with the `triggers` type.
- `triggers` may not target a table.

A.4.5 Identifier rules

A node identifier is formed as: prefix (`tbl_` or `fn_`) + name in `snake_case`. The display name is in `PascalCase`. This separation provides stable identifiers under name normalization.

A.5 Section 2: CGD Specification

A.5.1 Format

A single JSON object with two mandatory top-level keys: `functions` and `tables`.

A.5.2 `functions` block

The key is the function name in `PascalCase`. The value is an object with the fields of the YASF profile.

YASF profile fields for a function.

Field	Type	Required	Description
`purpose`	string	yes	One sentence describing what the function does
`processing`	array of string	yes	Ordered list of processing steps
`input`	object	yes	Contains `consumes` and `reads`
`input.consumes`	object	yes	Key: table name; value: what is consumed
`input.reads`	object	yes	Key: table name; value: what is read
`output`	object	yes	Contains `produces` and `writes`
`output.produces`	object	yes	Key: table name; value: what is created
`output.writes`	object	yes	Key: table name; value: what is written
`errors`	array	yes	Array of `{condition, result}` objects
`triggers`	array	yes	Array of `{function, condition}` objects
`contract`	object	yes	Six-component contract $C(f)$

Contract $C(f)$.

Component	Type	Description
`Cin`	array of string	Tables consumed by the function
`Cout`	array of string	Tables produced by the function
`R`	array of string	Tables read by the function
`W`	array of string	Tables written by the function
`Tin`	array of string	Functions that invoke the given function
`Tout`	array of string	Functions that the given function invokes

The empty set is denoted by an empty array `[]`. All contract arrays are treated as sets: element order is irrelevant and duplicates are forbidden. The contract may be given explicitly or derived from the graph. If both are present, they must match. The contract is considered closed if all six components are defined and correspond to the graph edges.

A.5.3 `tables` block

The key is the table name in `PascalCase`. The value is an object in the CGD Table Schema Profile format.

CGD Table Schema Profile.

The profile is based on JSON Schema with CGD-specific extensions using the `x-` prefix.

Mapping of CGD types to JSON Schema.

CGD type	JSON Schema type	Format	Note
`text`	`string`	—	
`number`	`number`	—	`integer` is a refinement
`bool`	`boolean`	—	
`date`	`string`	`date-time`	
`enum(values)`	`string`	—	+ `enum` field
`ref(table)`	matches target PK	matches target PK	+ `x-fk` field

Standard JSON Schema fields.

Field	Description
`properties`	Object describing the table fields
`required`	Array of required field names
`type`	Field type: `string`, `integer`, `number`, `boolean`
`format`	Field format: `uuid`, `email`, `date-time`, `uri`
`enum`	Array of admissible values for an enum field

CGD extensions.

Extensions are divided into two levels.
Table-level extensions (placed alongside `properties` and `required`):

Field	Required	Description
`x-kind`	yes	Table kind (see admissible values below)
`x-pk`	yes	Name of the primary-key field

Field-level extensions (placed inside `properties`):

Field	Required	Description
`x-fk`	no	Reference in the form `"Table.field"`
`x-unique`	no	Whether the field value must be unique

Table kinds (`x-kind`).

Kind	Meaning
`entity`	Primary domain object
`event`	Fact that occurred in the system
`reference`	Reference data
`log`	Action log
`projection`	Derived or aggregated table
`error`	Error description

Enum compatibility rule.

If a producer table writes an enum field and a consumer table reads it, then the following must hold: Allowed(producer) $\subseteq$ Allowed(consumer). The consumer must accept all values that the producer can create.

A.6 Naming rules

Element	Format	Examples
Function name	`PascalCase`, verb + noun	`RegisterUser`, `SendConfirmation`
Table name	`PascalCase`, singular noun	`User`, `AuditLog`
Mermaid node id	`snake_case` with prefix	`tbl_user`, `fn_register_user`
Table field name	`snake_case`	`id`, `email`, `created_at`
Enum value	`snake_case`	`active`, `pending`

A.7 Section consistency

A.7.1 Cross-sectional consistency: Mermaid and JSON

1. Every function from the `functions` block is present as an `fn_` node in Mermaid.
2. Every table from the `tables` block is present as a `tbl_` node in Mermaid.
3. There are no function nodes in Mermaid that are absent from `functions`.
4. There are no table nodes in Mermaid that are absent from `tables`.
5. The `kind` shown in a Mermaid table-node label matches the `x-kind` value in the `tables` block.
6. Every `consumes(T, F)` edge in Mermaid means that $T \in C_{\text{in}}(F)$.
7. Every `produces(F, T)` edge in Mermaid means that $T \in C_{\text{out}}(F)$.
8. Every `reads(T, F)` edge in Mermaid means that $T \in R(F)$.
9. Every `writes(F, T)` edge in Mermaid means that $T \in W(F)$.
10. Every `triggers(F1, F2)` edge in Mermaid means that $F2 \in T_{\text{out}}(F1)$ and $F1 \in T_{\text{in}}(F2)$.
11. There are no tables in contracts that are absent from the `tables` block.
12. There are no functions in `triggers` that are absent from the `functions` block.
13. The display name of a function in Mermaid matches the key in `functions` after name normalization.
14. The display name of a table in Mermaid matches the key in `tables` after name normalization.

A.7.2 Internal consistency of CGD Specification

15. The set of keys in `input.consumes` matches the set `Cin` in `contract`.
16. The set of keys in `input.reads` matches the set `R` in `contract`.
17. The set of keys in `output.produces` matches the set `Cout` in `contract`.
18. The set of keys in `output.writes` matches the set `W` in `contract`.
19. The set of values in `triggers[*].function` matches the set `Tout` in `contract`.

A.8 Representation layers

Layer	Format	Purpose
Model response	Mermaid + JSON	Structured output comparable across models
Canonical	`graph.json` + `tables.json`	Normalized source of truth
Visual	Mermaid render	Human-readable representation

The model responds at the first layer. Normalization maps it into the second. Visualization turns it into the third.

A.9 Scope boundaries

The standard provides structural comparability and verifiability of responses. It does not guarantee semantic correctness—the correctness of the chosen functions, tables, and links remains the responsibility of the author or the verifier.

A.10 Versioning

Version: `v1.3`. Date: `2026-03-15`.

Changes relative to v1.2: the type table was aligned with the article's formal core; `table-level` and `field-level` extensions were separated explicitly; internal JSON consistency checks were added (rules 15–19).

B Execution Profile

B.1 Response format

Return the result strictly in two sections. Text outside the sections is not allowed.

B.2 Section 1: Mermaid

Format: `flowchart LR`.

Tables are trapezoids with the `tbl_` prefix:

```
tbl_user[/"User (entity)"/]
```

Functions are rounded rectangles with the `fn_` prefix:

```
fn_register_user(["RegisterUser"])
```

Allowed edge types (must be specified in the edge label):

- `consumes`: table → function
- `produces`: function → table
- `reads`: table → function
- `writes`: function → table
- `triggers`: function → function

Forbidden: `table` → `table`, `function` → `function` without `triggers`.

B.3 Section 2: JSON

A single object with two keys: `functions` and `tables`.

B.3.1 `functions`

The key is the function name (`PascalCase`, verb + noun). The value is an object:

```
{
  "purpose": "one sentence",
  "processing": ["step 1", "step 2", "..."],
  "input": {
    "consumes": { "TableName": "short description" },
    "reads": { "TableName": "short description" }
  },
  "output": {
    "produces": { "TableName": "short description" },
    "writes": { "TableName": "short description" }
  },
  "errors": [
    { "condition": "condition", "result": "result" }
  ],
  "triggers": [
    { "function": "FunctionName", "condition": "condition" }
  ],
  "contract": {
    "Cin": ["tables"],
    "Cout": ["tables"],
    "R": ["tables"],
    "W": ["tables"],
    "Tin": ["functions"],
    "Tout": ["functions"]
  }
}
```

Rules:

- `consumes` / `reads` / `produces` / `writes`: the value must be a short string only, not an object.
- `processing`: each element must describe one action on one line.
- `contract`: arrays are treated as sets; order is irrelevant and duplicates are forbidden.
- Empty fields: `{}` for objects, `[]` for arrays.

B.3.2 `tables`

The key is the table name (`PascalCase`, singular). The value is an object:

```
{
  "x-kind": "entity | event | reference | log | projection | error",
  "x-pk": "field_name",
  "properties": {
    "field_name": { "type": "type", "format": "format" }
  },
```

```
  "required": ["required fields"]
}
```

Field-level extensions (inside `properties`):

- `"x-fk": "Table.field"` — foreign key.
- `"x-unique": true` — unique value.

`x-pk` and `x-fk` are `single-field` only.

Field types.

CGD type	JSON Schema type	Note
`text`	`string`	
`number`	`number`	`integer` is allowed
`bool`	`boolean`	
`date`	`string` + `format: date-time`	
`enum`	`string` + `enum: [values]`	
`ref`	target PK type + `x-fk`	

Field names and enum values: `snake_case`.

B.4 Section priority

If Mermaid and JSON diverge, JSON is the source of truth.

B.5 Consistency

The sections must be consistent:

- Every node in Mermaid = an entry in `functions` or `tables`.
- Every edge in Mermaid = an element of `contract`.
- Every element of `contract` = an edge in Mermaid.
- `kind` in a Mermaid table label = `x-kind` in `tables`.
- Keys in `consumes` / `reads` / `produces` / `writes` = the corresponding arrays in `contract`.
- Values in `triggers[*].function` = the `Tout` array in `contract`.

C ComputeSubgraphContract and transformation rules

C.1 Purpose

This appendix fixes a minimal formalization of the subgraph contract and the `ComputeSubgraphContract` algorithm used for decomposition, aggregation, and local verification of the structural realization.

C.2 Preconditions

Let the structural realization be given as a typed directed multigraph

$$G = (V, E, \mathrm{src}, \mathrm{dst}, \tau_V, \tau_E),$$

where $\tau_V(v) \in \{\texttt{function}, \texttt{table}\}$ and $\tau_E(e) \in \{\texttt{consumes}, \texttt{produces}, \texttt{reads}, \texttt{writes}, \texttt{triggers}\}$. Each function f has a contract

$$C(f) = \langle C_{\mathrm{in}}, C_{\mathrm{out}}, R, W, T_{\mathrm{in}}, T_{\mathrm{out}} \rangle,$$

where all components are fully derived from incident edges of the corresponding type.

C.3 Subgraph contract

For a chosen subgraph S, the set of its vertices is denoted by $V(S)$ and decomposed into two parts:

$$V(S) = V_{\mathrm{func}}(S) \cup V_{\mathrm{table}}(S),$$

where $V_{\mathrm{func}}(S)$ is the set of vertices of type `function` belonging to S, and $V_{\mathrm{table}}(S)$ is the set of vertices of type `table` belonging to S.

$$E(S) = \{e \in E(G) \mid \mathrm{src}(e) \in V(S) \vee \mathrm{dst}(e) \in V(S)\}$$

is the set of edges of graph G for which at least one endpoint belongs to $V(S)$.
For the subgraph S, the contract

$$C(S) = \langle C_{\mathrm{in}}(S), C_{\mathrm{out}}(S), R(S), W(S), T_{\mathrm{in}}(S), T_{\mathrm{out}}(S) \rangle$$

is defined. It describes only the external boundary of the subgraph: which tables and functions connect S to the rest of the graph.

C.4 `ComputeSubgraphContract` algorithm

```
function ComputeSubgraphContract(S):
    V_func(S) = {v in V(S) : t_V(v) = function}
    V_table(S) = {v in V(S) : t_V(v) = table}

    internal_tables = {t in V_table(S) :
        for all e in E where (src(e)=t or dst(e)=t) :
            (src(e) in V(S) and dst(e) in V(S))}

    interface_tables = V_table(S) \ internal_tables

    Cin(S) = {t in interface_tables : exists e in E(S),
              t_E(e) = consumes,
              src(e) = t, dst(e) in V_func(S)}

    Cout(S) = {t in interface_tables : exists e in E(S),
              t_E(e) = produces,
              src(e) in V_func(S), dst(e) = t}

    R(S) = {t in interface_tables : exists e in E(S),
```

```
            t_E(e) = reads,
            src(e) = t, dst(e) in V_func(S)}

W(S) = {t in interface_tables : exists e in E(S),
            t_E(e) = writes,
            src(e) in V_func(S), dst(e) = t}

Tin(S) = {f not in V(S) : exists e in E,
            t_E(e) = triggers,
            src(e) = f, dst(e) in V_func(S)}

Tout(S) = {f not in V(S) : exists e in E,
            t_E(e) = triggers,
            src(e) in V_func(S), dst(e) = f}

return <Cin(S), Cout(S), R(S), W(S), Tin(S), Tout(S)>
```

C.5 Note on interface tables

An interface table may belong to several components of the subgraph contract at once. For example, a table may be consumed by one boundary function and produced by another. This is a normal case: the subgraph contract fixes external relations rather than imposing a single role on a table.

C.6 Usage

C.6.1 Decomposition

Under decomposition, the node f is replaced with a subgraph S such that the following holds:

$$C(f) = C(S).$$

Intermediate internal tables are allowed as long as they do not reach the boundary of the subgraph.

C.6.2 Aggregation

Under aggregation, the subgraph S is collapsed into a single node f_S with the contract

$$C(f_S) = C(S).$$

This makes `ComputeSubgraphContract` the minimal formal mechanism that supports contract-preserving transformations between different levels of detail of the same structural realization.

D Task-driven context assembly

D.1 Purpose

This appendix fixes a draft formalization of task-driven context assembly as a mechanism for assembling context not from the entire dialogue history, but from the affected objects of the structural realization.

D.2 Goal

The goal of the mechanism is to obtain a compact and relevant context for the next task by using $R = \langle G, \Sigma, C \rangle$ as the single source of structural localization.

D.3 Input

The input consists of: a new task q; the current structural realization $R = \langle G, \Sigma, C \rangle$; and a history of decisions H, containing previous clarifications, accepted assumptions, and unresolved questions.

D.4 Localizing the task in R

First, the task q is localized in the structural realization. The result of this step is the initial set of affected objects:

$$A_0(q) = F_0 \cup T_0,$$

where F_0 are the functions explicitly mentioned or semantically corresponding to the task, and T_0 are the tables explicitly mentioned or directly connected to the functions found.

In practice, this means anchoring the task to the existing graph nodes, their contracts, and their signatures, rather than to the chat history as a whole.

D.5 Choosing the local neighborhood

After localization, a local structural neighborhood $N(A_0)$ is built. In the minimal version, it includes:

- the affected functions and tables themselves;
- the tables from C_{in}, C_{out}, R, W for the functions found;
- the functions from T_{in}, T_{out};
- neighboring nodes at graph distance one.

The idea is simple: not the entire graph enters the context, but only the nearest structural zone in which the change can realistically affect system behavior.

D.6 Lifting history by structural keys

After $N(A_0)$ has been built, the history H is lifted not in full, but only by the structural keys from the neighborhood found. The following are selected:

- past decisions explicitly related to functions and tables from $N(A_0)$;
- previously recorded blocking or advisory remarks concerning the same objects;
- accepted assumptions that have not yet been materialized in the current version of R.

Thus, history ceases to be an independent source of truth and becomes an auxiliary layer indexed by the objects of the structural realization.

D.7 Assembling task-specific context

The resulting task-specific context $\mathrm{CTX}(q)$ consists of three parts: the current state of the affected objects; the local structural neighborhood; and the relevant historical decisions lifted by the same keys. That is,

$$\mathrm{CTX}(q) = \mathrm{Current}(A_0 \cup N(A_0)) \cup \mathrm{History}(A_0 \cup N(A_0)).$$

Unrelated graph fragments and unrelated history are not included in the context, even if they are topically similar to the task.

D.8 Output

The output is a compact structurally anchored context suitable for: modifying R; repeated verification; and preparing the next model step.

D.9 Illustrative example

If the task is "add error logging for registration," the mechanism localizes it in the registration function, its error path, and the nearest logging sink. The history is then lifted by those same objects: it may turn out that a prior decision already existed to log errors separately from the success path, but that decision was never introduced into the current realization. The task-specific context therefore contains precisely this local zone rather than the full conversation and rather than the full system.

D.10 Status

In the current paper, task-driven context assembly is introduced as an architectural principle and a draft formalization. The full algorithm for neighborhood selection, ranking of historical fragments, and evaluation of context sufficiency remains future work.

E Demo run artifacts

This appendix lists the full model outputs used in Demos 1–3. Each file is included as ancillary material accompanying this paper.

`runs/scenario-01/claude-opus-4.6.md` Baseline run (Demo 1, no output contract). Model: Claude Opus 4.6. Contains the unstructured model response to the registration-system task.

`runs/scenario-02/gpt-5.4-thinking.md` Structured run (Demo 1 with output contract; Demo 2 without abstraction constraint; Demo 3 verification source). Model: GPT-5.4. Contains the full Mermaid graph, JSON specification, and function contracts.

`runs/scenario-02-top-level/gpt-5.4-thinking.md` Top-level run (Demo 2 with abstraction constraint). Model: GPT-5.4. Contains the reduced top-level graph with seven nodes and simplified contracts.

F YASF: the intent-interface specification

F.1 Purpose

YASF is a deterministic specification language used in the current CGD implementation as the form of interface-level intent representation I in the architectural pair $D = \langle I, R \rangle$. YASF describes a system in controlled natural language with a canonical set of sections. A YASF text is compiled into the structural realization $R = \langle G, \Sigma, C \rangle$, where G is a typed graph of functions and tables, Σ is a set of table signatures, and C is a set of function contracts. YASF plays the role of the authoring layer: it is the format in which a person fixes intent before it is translated into the formal core.

F.2 Design principles

Principle 1 — controlled natural language. YASF is written in natural language, but with a fixed section structure and controlled terminology. This yields a low entry threshold while preserving compilability.

Principle 2 — deterministic compilation. For the same consistent YASF text, the compiler produces the same structural realization $R = \langle G, \Sigma, C \rangle$ with the same graph relations, contracts, and signatures. Determinism is treated as a design invariant and is verified through conformance tests.

Principle 3 — minimality. YASF describes exactly what is required to build the structural realization: functions, tables, data flows, constraints, and contracts. Implementation details such as algorithms, data structures, and coding style are not part of YASF.

Principle 4 — modularity. Each function is described by a separate YASF block. Blocks may refer to one another through function names and table names. This enables independent development, verification, and compilation of separate parts of intent.

Principle 5 — traceability. Every element of the structural realization—a node or edge in G, a signature $\Sigma(t)$, or a contract $C(f)$—must be traceable to a specific YASF section. Conversely, each YASF section must produce specific elements in $R = \langle G, \Sigma, C \rangle$.

F.3 Structure of a YASF block

A YASF block describes one function. Its canonical structure includes the following sections.

Purpose — a short natural-language description of the function (1–2 sentences). Compiles into: a `function` node in graph G with a name and description.

Input data — the list of tables consumed by the function, indicating fields, types, keys, and `kind`. Compiles into: `table` nodes in G, `consumes` edges, the C_{in} component of the contract, and signatures $\Sigma(t)$ of input tables.

Reference data — the list of tables read by the function for reference. Compiles into: `reads` edges, the R component of the contract, and—when needed—the signatures of the corresponding tables.

Processing — the sequence of processing steps in controlled natural language. The order of items gives the basic sequence of steps. The concrete graph edges and contract dependencies are formed during compilation jointly with table signatures and the other sections of the block.

Output data — the list of tables produced by the function, indicating fields, types, keys, and `kind`. Compiles into: `produces` edges, the C_{out} component of the contract, and signatures $\Sigma(t)$ of output tables.

Errors — the list of error tables produced by the function when it deviates from the success path. Compiles into: `produces` edges, additional C_{out} elements, and signatures $\Sigma(t)$ of error

tables (`kind: error`).

Side effects — the list of tables written by the function that are not its main outputs. Compiles into: `writes` edges, the W component of the contract, and when needed the signatures of the corresponding tables.

Triggers — the list of functions activated by the given function on completion. Compiles into: `triggers` edges and the T_{out} component of the contract.

Contract — an explicit fixation of the six-component contract $C(f) = \langle C_{\text{in}}, C_{\text{out}}, R, W, T_{\text{in}}, T_{\text{out}} \rangle$. It may be written manually by the author or generated by the compiler from the other sections. If both are present, the compiler checks consistency.

F.4 Compilation rules: YASF $\to R = \langle G, \Sigma, C \rangle$

Rule 1 — node extraction. Every function name mentioned in the "Purpose" or "Triggers" sections produces a node with $\tau_V =$ `function`. Every table name mentioned in "Input data," "Reference data," "Output data," "Errors," or "Side effects" produces a node with $\tau_V =$ `table`.

Rule 2 — edge extraction. A table from "Input data" produces a `consumes` (table $\to$ function) edge. A table from "Output data" or "Errors" produces a `produces` (function $\to$ table) edge. A table from "Reference data" produces a `reads` (table $\to$ function) edge. A table from "Side effects" produces a `writes` (function $\to$ table) edge. A function from "Triggers" produces a `triggers` (function $\to$ function) edge.

Rule 3 — signature extraction. For every table described with fields, types, and keys, a signature $\Sigma(t) = \langle \text{Fields}, \text{Keys}, \text{Kind}, \text{Constraints} \rangle$ is formed according to the definition of the paper's formal core.

Rule 4 — contract formation. The contract $C(f)$ is formed from the extracted edges. If the author explicitly specified a contract in the "Contract" section, the compiler checks equality between the generated and author-specified contract. A mismatch is treated as a structural defect.

Rule 5 — completeness checking. After a block is compiled, the following is checked: all tables appearing in the contract have defined signatures, all edges are type-correct, no nodes are left without edges, and the resulting local structural realization is consistent with the verification profile of the current step.

F.5 Mapping between YASF sections and contract components

YASF section	Contract	Edge type	Direction
Input data	C_{in}	consumes	table $\to$ function
Reference data	R	reads	table $\to$ function
Output data	C_{out}	produces	function $\to$ table
Errors	C_{out}	produces	function $\to$ table
Side effects	W	writes	function $\to$ table
Triggers	T_{out}	triggers	function $\to$ function
External block	T_{in}	triggers	function $\to$ function

The T_{in} component has no dedicated section inside a function block: it is derived from the "Triggers" sections of other blocks that refer to this function.

F.6 Example of a YASF block

```
Function: RegisterUser
Purpose: Register a new user by email.
```

```
Input data:
RegistrationRequest (id PK, name: string required,
  email: string required, timestamp: timestamp required;
  kind: event)

Reference data:
User (id PK, name: string, email: string, status: string,
  created_at: timestamp; kind: entity)
  -- read to check email uniqueness.

Processing:
1. Accept RegistrationRequest.
2. Check format and completeness of the data.
3. Check email uniqueness in the User table.
4. Create a User record with status pending.
5. Record RegistrationEvent.
6. Write the action to AuditLog.
7. Trigger SendConfirmation.

Output data:
User (id PK, name: string, email: string, status: string,
  created_at: timestamp; kind: entity)
RegistrationEvent (id PK, user_id FK->User,
  event_type: string, timestamp: timestamp; kind: event)

Errors:
RegistrationError (id PK, error_code: string,
  message: string, timestamp: timestamp; kind: error)
  -- for invalid input or duplicate email.

Side effects:
AuditLog (id PK, action: string, entity_id: string,
  timestamp: timestamp; kind: log)

Triggers:
SendConfirmation
  -- send registration confirmation.

Contract v2:
C(RegisterUser) = <Cin: {RegistrationRequest},
  Cout: {User, RegistrationEvent, RegistrationError},
  R: {User}, W: {AuditLog},
  Tin: empty, Tout: {SendConfirmation}>
```

F.7 Decomposition in YASF

When a function is decomposed into subfunctions, each subfunction receives its own YASF block. The original block is preserved as an aggregated representation with a note such as:

```
Function: RegisterUser (aggregated node)
Decomposition: ValidateInput -> CheckUniqueness ->
  CreateUser -> (triggers) SendConfirmation
Refinement map: see blocks ValidateInput,
```

```
CheckUniqueness, CreateUser, SendConfirmation
```

Each subfunction block has the full structure from §F.3. Intermediate tables (`ValidatedRequest`, `UniqueRequest`) are described in the "Output data" and "Input data" sections of the corresponding subfunctions. Formal computation of the external contract of the subgraph in such a decomposition is moved to Appendix C.

F.8 Verification-driven dialogue in YASF

When the verifier discovers defects after compiling a YASF block, they are fixed and classified as follows.

Blocking — require the author to extend the YASF text. Example: the "Errors" section is absent or empty while the "Processing" section contains branching conditions. The verifier generates a question; after the author answers, the section is extended.

Auto-resolvable — the compiler or verifier can extend the block automatically. Example: if an error path has been added and the function already has an `AuditLog` side effect, an `AuditLog` entry on the error path is added by analogy.

Advisory — warnings recorded as comments on the YASF block. Example: a table from "Output data" is not consumed by any known function.

After all blocking defects have been resolved and auto-resolvable fixes have been applied, the block is recompiled into $R = \langle G, \Sigma, C \rangle$ and checked again until specification closure is reached (see §5 of the main text).

F.9 Conformance tests

Compilation determinism is verified through conformance tests—a fixed set of pairs:

Input	**Expected output**
YASF text (function block)	Subgraph G: nodes, edges, types
YASF text (function block)	Contract $C(f)$: six components
YASF text (table description)	Signature $\Sigma(t)$: Fields, Keys, Kind, Constraints
YASF text (full block)	Local structural realization $R = \langle G, \Sigma, C \rangle$

A test is considered passed if the compiler produces a structurally identical local realization, up to ordering of elements in sets. The conformance-test set is extended as new YASF constructs appear.

F.10 Limitations of the current YASF version

Not supported: a formal grammar (YASF is defined through section structure and compilation rules rather than through BNF/EBNF), automatic semantic processing of free text inside the "Processing" section (the compiler relies on mentioned function and table names rather than on the full semantics of free text), inheritance between blocks (one YASF block cannot extend another), and conditional sections (branching inside a block is not formalized; the error path is described through a separate "Errors" section).

Planned: a formal grammar for YASF (to enable automatic syntax validation), conditional sections (to describe explicit branching inside a block), and template support (typical block patterns that can be inherited and overridden).

F.11 Cross-references

From Appendix F	Reference
Rules 1–2 (node and edge extraction)	§4.1 (graph G and edge types)
Rule 3 (signature extraction)	§4.2 (table signatures Σ)
Rule 4 (contract formation)	§4.3 (function contracts C)
Rule 5 (completeness checking)	§5.4 (specification closure)
Decomposition	Appendix C (ComputeSubgraphContract)
Verification-driven dialogue	§§5.3–5.4
Conformance tests	§3.1 (YASF as the intent interface)

References

[1] Anthropic. Best practices for Claude Code. `https://code.claude.com/docs/en/best-practices`, 2026. Accessed: 2026-03-16.

[2] Augment Code. Build with intent. `https://www.augmentcode.com/product/intent`, 2026. Accessed: 2026-03-16.

[3] Andrey Breslav. Codespeak: Software engineering with AI. `https://codespeak.dev/`, 2026. Accessed: 2026-03-16.

[4] Cursor. Rules documentation. `https://cursor.com/docs/rules`, 2026. Accessed: 2026-03-16.

[5] GitHub. Spec-driven development with AI: Get started with a new open source toolkit. `https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/`, 2025. GitHub Blog, September 2, 2025. Accessed: 2026-03-16.

[6] OpenAI. Symphony. `https://github.com/openai/symphony`, 2026. Open-source, Apache 2.0. Accessed: 2026-03-16.

[7] Deepak Babu Piskala. Spec-driven development: From code to contract in the age of AI coding assistants. *arXiv preprint*, 2026.

Supplementary materials. Appendices, JSON schemas, and examples are available at `https://github.com/alexfdo/cgd-paper`.

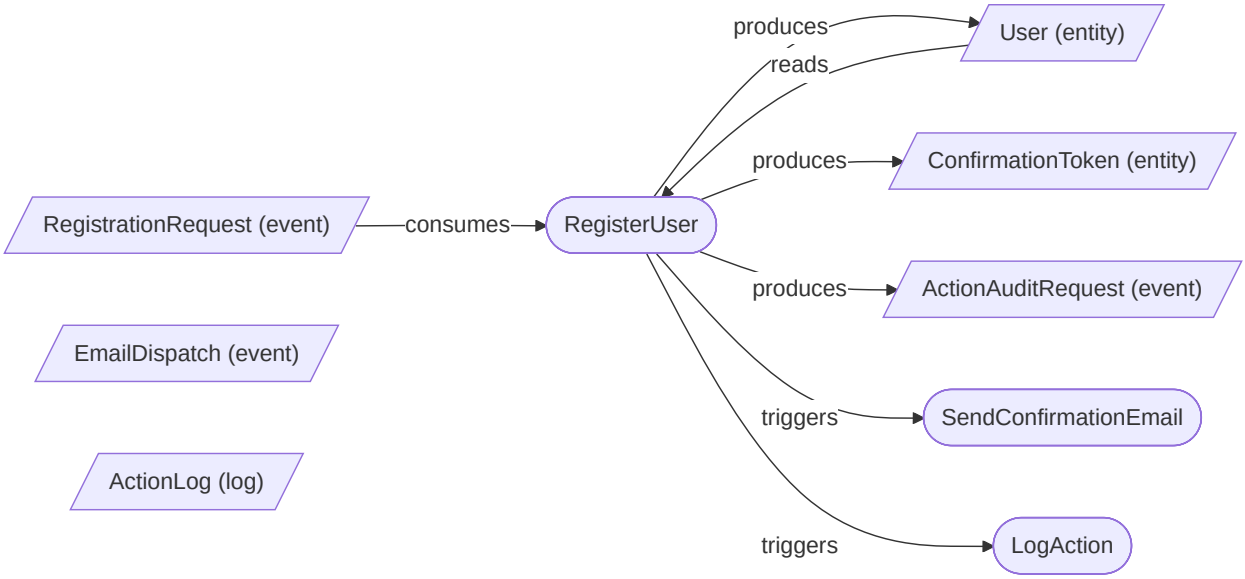

Figure 1: Structured GPT-5.4 graph projection with output contract (Demo 1).

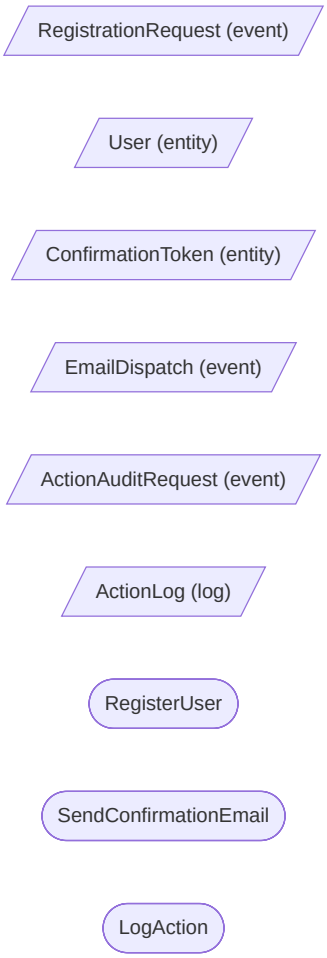

Figure 2: Structured GPT-5.4 graph without abstraction constraint (Demo 2).

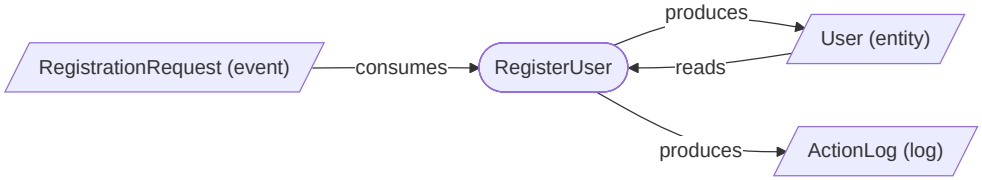

Figure 3: Top-level GPT-5.4 graph with abstraction constraint (Demo 2).